

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

по лабораторной работе №10

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Поиск расстояний во взвешенном графе»

Выполнили:

студенты группы 24ВВВЗ

Задоркин Тимофей

Комиссаров Андрей

Приняли:

Юрова О.В.

Деев М.В.

Пенза 2025

Цель работы – ознакомиться с алгоритмом поиска расстояний во взвешенном графе и освоить его практическую реализацию

Общие сведения.

Во взвешенном графе в отличие от не взвешенного каждое ребро имеет вес, отличный от нуля. Поэтому в матрице смежности взвешенного графа содержится информация не только о наличии ребра, но и о его весе.

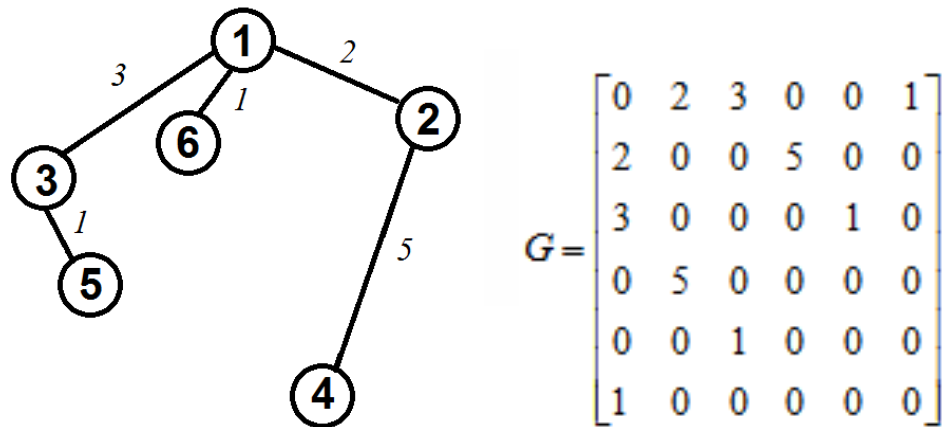


Рисунок 1 – Граф

Поиск расстояний между вершинами в таком графе также возможно построить используя процедуры обхода графа. Отличие от поиска расстояний в не взвешенном графе будет состоять в том, что при обновлении расстояния до вершины при ее посещении оно будет увеличиваться не на 1, а на величину веса ребра.

Таким образом, можно предложить следующую реализацию алгоритма обхода в ширину.

Вход: G – матрица смежности графа, v – исходная вершина.

Выход: DIST – вектор расстояний до всех вершин от исходной.

Алгоритм ПОШ

- 1.1. для всех i положим $DIST[i] = -1$ пометим как "не посещенную";
- 1.2. **ВЫПОЛНЯТЬ** BFS(v).
- 1.3 для всех i вывести $DIST[i]$ на экран;

Алгоритм BFS(v):

- 2.1. Создать пустую очередь $Q = \{\}$;
- 2.2. Поместить v в очередь $Q.push(v)$;
- 2.3. Обновить вектор расстояний $DIST[x] = 0$;
- 2.4. **ПОКА** $Q \neq \emptyset$ очередь не пуста **ВЫПОЛНЯТЬ**
- 2.5. $v = Q.front()$ установить текущую вершину;
- 2.6. Удалить первый элемент из очереди $Q.pop()$;
- 2.7. вывести на экран v ;
- 2.8. **ДЛЯ** $i = 1$ **ДО** $size_G$ **ВЫПОЛНЯТЬ**
- 2.9. **ЕСЛИ** $G(v,i) > 0$ **И** $DIST[i] = -1$
- 2.10. **ТО**
- 2.11. Поместить i в очередь $Q.push(i)$;
- 2.12. Обновить вектор расстояний $DIST[i] = DIST[v] + G(v,i)$;

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного взвешенного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При

реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.

3.* Сгенерируйте (используя генератор случайных чисел) матрицу смежности для ориентированного взвешенного графа G . Выведите матрицу на экран и осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием.

Задание 2

1. Для каждого из вариантов сгенерированных графов (ориентированного и не ориентированного) определите радиус и диаметр.
2. Определите подмножества периферийных и центральных вершин.

Результат работы программы

```
Введите количество вершин: 7
Тип графа (0 - неориентированный, 1 - ориентированный): 0

Матрица смежности:
0 1 0 0 0 1 0
1 0 1 0 1 0 0
0 1 0 1 1 1 1
0 0 1 0 0 1 1
0 1 1 0 0 0 1
1 0 1 1 0 0 1
0 0 1 1 1 1 0

Матрица весов:
0 3 0 0 0 10 0
3 0 9 0 9 0 0
0 9 0 2 4 1 5
0 0 2 0 0 8 1
0 9 4 0 0 0 1
10 0 1 8 0 0 7
0 0 5 1 1 7 0
```

Рисунок 1-1 – 7 вершин неориентированный граф

```

Введите стартовую вершину: 0

=== Алгоритм Дейкстры для взвешенного графа ===
Кратчайшие расстояния от вершины 0:
  до вершины 0: 0
  до вершины 1: 3
  до вершины 2: 11
  до вершины 3: 13
  до вершины 4: 12
  до вершины 5: 10
  до вершины 6: 13

Матрица кратчайших расстояний между всеми вершинами:
  0  3  11  13  12  10  13
  3  0   9  11   9  10  10
 11  9   0   2   4   1   3
 13 11   2   0   2   3   1
 12  9   4   2   0   5   1
 10 10   1   3   5   0   4
 13 10   3   1   1   4   0

=== МЕТРИКИ ГРАФА ===

Эксцентриситеты вершин:
  Вершина 0: 13
  Вершина 1: 11
  Вершина 2: 11
  Вершина 3: 13
  Вершина 4: 12
  Вершина 5: 10
  Вершина 6: 13

Радиус графа: 10
Диаметр графа: 13
Центральные вершины (1): 5
Периферийные вершины (3): 0 3 6

```

Рисунок 1-2 – 7 вершин неориентированный граф

```

Введите количество вершин: 7
Тип графа (0 – неориентированный, 1 – ориентированный): 1

Матрица смежности:
  0  1  1  0  1  0  1
  1  0  1  0  1  1  0
  0  0  0  1  0  0  1
  0  0  1  0  0  0  0
  0  0  1  0  0  0  0
  1  1  0  1  1  0  1
  1  0  0  1  1  0  0

Матрица весов:
  0  4  4  0  7  0 10
  7  0  6  0  2  7  0
  0  0  0  8  0  0  9
  0  0 10  0  0  0  0
  0  0  6  0  0  0  0
 10  7  0  2  1  0  3
  8  0  0  6  7  0  0

```

Рисунок 2-1 – 7 вершин ориентированный граф

Введите стартовую вершину: 0

=== Алгоритм Дейкстры для взвешенного графа ===

Кратчайшие расстояния от вершины 0:

до вершины 0: 0
до вершины 1: 4
до вершины 2: 4
до вершины 3: 12
до вершины 4: 6
до вершины 5: 11
до вершины 6: 10

Матрица кратчайших расстояний между всеми вершинами:

0	4	4	12	6	11	10
7	0	6	9	2	7	10
17	21	0	8	16	28	9
27	31	10	0	26	38	19
23	27	6	14	0	34	15
10	7	7	2	1	0	3
8	12	12	6	7	19	0

=== МЕТРИКИ ГРАФА ===

Эксцентриситеты вершин:

Вершина 0: 12
Вершина 1: 10
Вершина 2: 28
Вершина 3: 38
Вершина 4: 34
Вершина 5: 10
Вершина 6: 19

Радиус графа: 10

Диаметр графа: 38

Центральные вершины (2): 1 5

Периферийные вершины (1): 3

Рисунок 2-2 – 7 вершин ориентированный граф

Вывод: в ходе выполнения лабораторной работы ознакомились с алгоритмом поиска расстояний во взвешенном графе и освоили его практическую реализацию.

Приложение – Листинг программы

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>
#include <time.h>
#include <queue>
#include <limits.h>
#include <windows.h>
#include <vector>

using namespace std;

void Dijkstra_Matrix(int** G, int** W, int numG, int* DIST, int
start) {
    for (int i = 0; i < numG; i++) {
        DIST[i] = INT_MAX;
    }
    DIST[start] = 0;

    vector<bool> visited(numG, false);

    for (int count = 0; count < numG; count++) {
        int min_dist = INT_MAX;
        int u = -1;

        for (int i = 0; i < numG; i++) {
            if (!visited[i] && DIST[i] < min_dist) {
                min_dist = DIST[i];
                u = i;
            }
        }

        if (u == -1 || DIST[u] == INT_MAX) break;

        visited[u] = true;

        for (int v = 0; v < numG; v++) {
            if (G[u][v] > 0 && !visited[v] && DIST[u] !=
INT_MAX) {
                int new_dist = DIST[u] + W[u][v];
                if (new_dist < DIST[v]) {
                    DIST[v] = new_dist;
                }
            }
        }
    }
}

int calculateEccentricity(int** distances, int numG, int vertex)
{
    int max_dist = 0;
```

```

    bool all_reachable = true;

    for (int i = 0; i < numG; i++) {
        if (vertex == i) continue;

        if (distances[vertex][i] == INT_MAX) {
            all_reachable = false;
            continue;
        }

        if (distances[vertex][i] > max_dist) {
            max_dist = distances[vertex][i];
        }
    }

    if (!all_reachable) {
        return -1;
    }

    return max_dist;
}

void calculateGraphMetrics(int** distances, int numG, int*
radius, int* diameter,
    int* central_vertices, int* peripheral_vertices, int*
central_count,
    int* peripheral_count, int** eccentricities_array) {

    int* eccentricities = (int*)malloc(numG * sizeof(int));

    for (int i = 0; i < numG; i++) {
        eccentricities[i] = calculateEccentricity(distances,
numG, i);
    }

    if (eccentricities_array != NULL) {
        *eccentricities_array = eccentricities;
    }

    bool connected = true;
    for (int i = 0; i < numG; i++) {
        if (eccentricities[i] == -1) {
            connected = false;
            break;
        }
    }

    if (!connected) {
        printf("Граф не является связным! Метрики не
определены.\n");
        *radius = -1;
        *diameter = -1;
        *central_count = 0;
    }
}

```

```

        *peripheral_count = 0;
        if (eccentricities_array == NULL) {
            free(eccentricities);
        }
        return;
    }

    *radius = INT_MAX;
    *diameter = 0;

    for (int i = 0; i < numG; i++) {
        if (eccentricities[i] < *radius) {
            *radius = eccentricities[i];
        }
        if (eccentricities[i] > *diameter) {
            *diameter = eccentricities[i];
        }
    }

    *central_count = 0;
    *peripheral_count = 0;

    for (int i = 0; i < numG; i++) {
        if (eccentricities[i] == *radius) {
            central_vertices[(*central_count)++] = i;
        }
        if (eccentricities[i] == *diameter) {
            peripheral_vertices[(*peripheral_count)++] = i;
        }
    }

    if (eccentricities_array == NULL) {
        free(eccentricities);
    }
}

void printMatrix(int** G, int numG) {
    printf("\nМатрица смежности:\n");
    for (int i =

        0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            printf("%3d", G[i][j]);
        }
        printf("\n");
    }
}

void printWeightMatrix(int** W, int numG) {
    printf("\nМатрица весов:\n");
    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {

```

```

        printf("%3d", W[i][j]);
    }
    printf("\n");
}

}

void initDIST(int* DIST, int numG) {
    for (int i = 0; i < numG; i++) {
        DIST[i] = INT_MAX;
    }
}

void printDIST(int* DIST, int numG, int start) {
    printf("Кратчайшие расстояния от вершины %d:\n", start);
    for (int i = 0; i < numG; i++) {
        printf("    до вершины %d: ", i);
        if (DIST[i] == INT_MAX) {
            printf("Недостижима\n");
        }
        else {
            printf("%d\n", DIST[i]);
        }
    }
}

void printAllDistances(int** distances, int numG) {
    printf("\nМатрица кратчайших расстояний между всеми
вершинами:\n");
    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            if (distances[i][j] == INT_MAX) {
                printf("    ∞");
            }
            else {
                printf("%4d", distances[i][j]);
            }
        }
        printf("\n");
    }
}

void printEccentricities(int* eccentricities, int numG) {
    printf("\nЭксцентриситеты вершин:\n");
    for (int i = 0; i < numG; i++) {
        if (eccentricities[i] == -1) {
            printf("    Вершина %d: не определена (граф
несвязный)\n", i);
        }
        else {
            printf("    Вершина %d: %d\n", i, eccentricities[i]);
        }
    }
}

```

```
int main() {
    system("chcp 1251");
    int** G;
    int** W;
    int numG, start;
    int* DIST;
    int directed;

    printf("Введите количество вершин: ");
    scanf("%d", &numG);

    printf("Тип графа (0 - неориентированный, 1 - ориентированный): ");
    scanf("%d", &directed);

    DIST = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));
    W = (int**)malloc(numG * sizeof(int*));
    for (int i = 0; i < numG; i++) {
        G[i] = (int*)malloc(numG * sizeof(int));
        W[i] = (int*)malloc(numG * sizeof(int));
    }

    srand(time(NULL));

    for (int i = 0; i < numG; i++) {
        for (int j = 0; j < numG; j++) {
            if (i == j) {
                G[i][j] = 0;
                W[i][j] = 0;
            }
            else {
                if (directed) {
                    G[i][j] = rand() % 2;
                    if (G[i][j] == 1) {
                        W[i][j] = rand() % 10 + 1;
                    }
                }
                else {
                    W[i][j] = 0;
                }
            }
            else {
                if (i > j) {
                    G[i][j] = G[j][i];
                    W[i][j] = W[j][i];
                }
                else {
                    G[i][j] = rand() % 2;
                    if (G[i][j] == 1) {
                        W[i][j] = rand() % 10 + 1;
                    }
                }
            }
        }
    }
}
```

```

        W[i][j] = 0;
    }
}

printMatrix(G, numG);
printWeightMatrix(W, numG);

printf("\nВведите стартовую вершину: ");
scanf("%d", &start);

printf("\n=== Алгоритм Дейкстры для взвешенного графа ===\n");
initDIST(DIST, numG);
Dijkstra_Matrix(G, W, numG, DIST, start);
printDIST(DIST, numG, start);

int** all_distances = (int**)malloc(numG * sizeof(int*));
for (int i = 0; i < numG; i++) {
    all_distances[i] = (int*)malloc(numG * sizeof(int));
    initDIST(all_distances[i], numG);
    Dijkstra_Matrix(G, W, numG, all_distances[i], i);
}

printAllDistances(all_distances, numG);

int radius, diameter;
int* central_vertices = (int*)malloc(numG * sizeof(int));
int* peripheral_vertices = (int*)malloc(numG * sizeof(int));
int central_count, peripheral_count;
int* eccentricities = NULL;

calculateGraphMetrics(all_distances, numG, &radius,
&diameter,
central_vertices, peripheral_vertices, &central_count,
&peripheral_count, &eccentricities);

printf("\n=== МЕТРИКИ ГРАФА ===\n");
if (radius == -1) {
    printf("Граф не является связным. Метрики не определены.\n");
}
else {
    printEccentricities(eccentricities, numG);

    printf("\nРадиус графа: %d\n", radius);
    printf("\nДиаметр графа: %d\n", diameter);
}

```

```

    printf("Центральные вершины (%d): ", central_count);
    for (int i = 0; i < central_count; i++) {
        printf("%d ", central_vertices[i]);
    }
    printf("\n");

    printf("Периферийные вершины (%d): ", peripheral_count);
    for (int i = 0; i < peripheral_count; i++) {
        printf("%d ", peripheral_vertices[i]);
    }
    printf("\n");
}

free(DIST);
for (int i = 0; i < numG; i++) {
    free(G[i]);
    free(W[i]);
    free(all_distances[i]);
}
free(G);
free(W);
free(all_distances);
free(central_vertices);
free(peripheral_vertices);
if (eccentricities != NULL) {
    free(eccentricities);
}

printf("\nНажмите любую клавишу для выхода...");
_getch();
return 0;
}

```